Study Buddy Enhanced

LSDTFH[1]

University of the Witwatersrand

August 5, 2025 – September 2, 2025

Project Team	
Author Name	Student Number
Lebo Kharafu	2577390
TrustGod Mokgwadi	2705185
Favour Mtshali	2662283
Dimpho Matea	2678460
Sinethemba Khumalo	2672254
Author Six	678901
Hulisani Netshaulu	2769364

Contents

Chapter 1

Project Road-map

1.1 Hidden Requirements

Filter by courses
Send notifications
Link similar courses

1.2 Development Phases

The project follows four iterative sprints aligned with course requirements:

1.2.1 Sprint 1 (Foundation)

- **Focus**: System setup and core infrastructure
- **Key Deliverables**:
  - Authentication system (Google OAuth)
  - Version control implementation
  - Documentation site (VitePress)
  - Project management framework (Crystal Clear)
- **Rubric Weight**: 80% of sprint grade

1.2.2 Sprint 2 (Core Features)

- **Focus**: Implementation of primary functionality
- **Key Deliverables**:
  - Course specification system
  - Buddy matching algorithm

  - Resource sharing with PDF tagging
  - Google Calendar integration

- **Rubric Weight**: 95% of sprint grade

1.2.3 Sprint 3 (Feature Completion)

- **Focus**: Enhanced functionality and refinement
- **Key Deliverables**:
  - Discussion threads with Markdown
  - Similar course tagging
  - Progress tracking dashboard
  - Accessibility improvements

1.2.4 Sprint 4 (Polishing)

- **Focus**: Final touches and documentation
- **Key Deliverables**:
  - Comprehensive testing coverage
  - Group and individual reports
  - Final presentation materials
  - Stretch goals (if time permits)

1.3 Feature Prioritization

Table 1.1: Feature Priority by Sprint

Priority	**Sprint**	**Key Features**
Critical	1	Auth, Docs, Setup
High	2	Courses, Matching, Resources
Medium	3	Discussions, Progress
Low	4	Offline support

1.4 Technical Implementation

1.4.1 Core Stack

- **Frontend**: React with TypeScript

- **Backend**: Node.js/Express
- **Database**: PostgreSQL (NeonDB) + Sequalise
- **Testing**: Cypress + Vitest

1.4.2 Integration Points

- Google OAuth for authentication
- Google Calendar API for scheduling
- Socket.IO for real-time chat
- Firebase for notifications

1.5 Risk Management

- **Technical Risks**: API rate limits, database scaling
- **Schedule Risks**: Feature creep in Sprint 2
- **Mitigation Strategies**:
  - Weekly client check-ins
  - Automated testing coverage
  - Priority-based backlog grooming

Chapter 2

Project Management

2.1 Overview

Crystal Clear is the simplest and lightest member of the Crystal Methods family, designed for small teams (typically 16 people) working on low-criticality software projects, such as web applications or prototypes. It emphasizes people, communication, and minimal overhead to deliver working software quickly. This methodology is particularly well-suited for academic or small-scale commercial projects where flexibility and rapid iteration are valued over rigorous, heavyweight processes. By focusing on core agile properties and lightweight practices, Crystal Clear enables teams to maintain productivity and adapt to changes efficiently.

2.2 Methodology

Crystal Clear is built upon seven core properties that ensure project success:

1. **Frequent Delivery**: Deliver working software increments every 13 months (or more frequently for smaller features) to gather user feedback.
2. **Reflective Improvement**: Hold regular retrospectives to assess processes and implement improvements.
3. **Close Communication**: Encourage frequent and direct communication among team members, whether in-person or via digital means.
4. **Personal Safety**: Foster an environment where team members feel safe to express ideas and concerns without fear of blame.
5. **Focus**: Ensure team members have clear tasks and dedicated time to complete them without interruptions.
6. **Easy Access to Users**: Maintain regular contact with end-users to validate features and usability.

7. **Technical Environment**: Use supportive tools like version control, automated testing, and continuous integration.

Key Roles

- **Sponsor**: Provides funding and high-level direction.
- **Senior Designer-Programmer**: Leads technical decisions and mentors others.
- **Designer-Programmers**: Develop features and implement solutions.
- **Business Experts/Users**: Offer domain knowledge and end-user feedback.

Key Practices

Lightweight practices include:

- Frequent delivery of usable code.
- Reflective improvement via retrospectives.
- Osmotic communication through co-location or digital proximity.
- Use of task boards (e.g., Trello) for priority management.
- Regular user feedback sessions.
- Automated testing and version control.

Deliverables

Essential work products include:

- Release sequence (high-level feature timeline)
- User stories/use cases
- Risk list (prioritized risks and mitigations)
- Iteration plan (short-term task listing)
- Design notes (key technical decisions)
- Working software (primary output)
- Test cases (automated validation)

2.3 Why Crystal Clear is Recommended

Crystal Clear is ideally suited for small teams developing low-criticality software, such as web applications or prototypes, for the following reasons:

- **Small Team Focus**: Designed for teams of 16 people, making it scalable for academic or startup environments.
- **Low Criticality**: Suitable for projects where failures do not result in significant harm, such as educational tools or game prototypes.
- **Flexibility**: Allows teams to prioritize coding and collaboration over bureaucratic processes, aligning well with iterative development common in web projects using HTML, JavaScript, or Python.
- **Beginner-Friendly**: Easy to adopt, even for teams new to agile methodologies, due to its minimalistic and intuitive practices.
- **Emphasis on Communication**: Promotes frequent and open dialogue, reducing misunderstandings and accelerating problem-solving.

2.4 Industry Resources & Justification

- Agile Alliance Glossary - Crystal: The Agile Alliance, a premier global agile community, provides an overview of Crystal, affirming its status as a recognized and legitimate agile methodology.
- ScienceDirect Topic on Crystal Method: An academic resource that discusses Crystal's place in software engineering, highlighting its human-powered and adaptive nature, which justifies its recommendation for small teams.
- Agile Business Consortium - Crystal Clear: Provides a concise breakdown of the methodology's key elements, supporting the practical steps for implementation outlined in this chapter.
- VersionOne Agile 101 - Crystal Method: VersionOne (now part of CollabNet VersionOne) was a leading vendor in agile lifecycle management tools. Their guide reinforces the suitability of Crystal for small, co-located teams working on low-criticality projects.

Chapter 3

Technology Stack

3.1 Frontend Stack

The frontend of the system is designed to provide a responsive, interactive, and user-friendly interface. It emphasizes modularity, reusability, and maintainability of components while ensuring fast load times and smooth navigation.

3.1.1 Framework and Libraries

- **React.js** — Used as the primary framework for building a component-based user interface. React enables efficient rendering through its virtual DOM and promotes reusability of UI components.
- **Vite** — Adopted as the build tool and development server. Vite provides a faster development experience with hot module replacement and optimized builds.
- **CSS** — Utilized for styling to achieve a clean, modern, and responsive design with utility-first classes.
- **Lucide Icons** — Used for lightweight, customizable icons across the interface.

3.1.2 Design Principles

- **Responsiveness** — Ensures seamless user experiences across devices with different screen sizes.
- **Accessibility (aria)** — Follows best practices to make the application usable for all users, including those with disabilities.
- **Performance Optimization** — Lazy loading, code splitting, and optimized assets improve loading times.

3.1.3 Testing and Quality Assurance

- **Playwright** — Applied for integration testing of whole app to guarantee reliability.

3.1.4 Deployment

- **Vercel** — The frontend is deployed on Vercel, leveraging its CDN for fast global delivery and integration with GitHub for continuous deployment.

3.1.5 Industry Resources

To justify the frontend technology choices, several authoritative industry resources were consulted:

- **JavaScript (JS)**: Recognized as the most widely used programming language for web development, according to the Stack Overflow Developer Survey. `https://survey.stackoverflow.co/2024/`
- **React**: Maintained by Meta and widely adopted in industry, React is considered a standard for building scalable, component-based user interfaces. `https://react.dev/`
- **Vite**: A next-generation frontend tooling that provides fast Hot Module Replacement (HMR) and optimized builds, recommended by many open-source contributors and adopted across projects for its developer experience. `https://vitejs.dev/`
- **CSS**: CSS is simple to use, and what was familiar to the group. No hassle, no need to learn syntax making the whole design experience seamless. `https://www.w3schools.com/css/`

3.2 Backend Stack

Overview: This document provides a comprehensive guide to setting up and running the SDP Project backend API. The backend is built with Node.js, Express, Sequelize ORM, and PostgreSQL (NeonDB).

TECHNOLOGY STACK FOR BACKEND

- **Runtime:** Node.js
- **Framework:** Express.js
- **Database:** PostgreSQL (NeonDB)
- **ORM:** Sequelize
- **Documentation:** Swagger

- **Authentication:** JWT (JSON Web Tokens)
- **Development Tool:** Nodemon

PREREQUISITES

- **Node.js:** v16 or higher
- **Package Manager:** npm
- **Database:** PostgreSQL (NeonDB recommended)
- **Version Control:** Git

3.2.1 Database System

Option A: Local PostgreSQL

- **Install PostgreSQL locally**
- **Create a database**
- **Update `.env` with local credentials**

Option B: NeonDB (Recommended)

- **Sign up at `https://neon.tech`**
- **Create a new project**
- **Copy the connection string**
- **Update `.env` with NeonDB credentials**

3.2.2 Industry Resources

BACKEND References used:

1. `https://neon.com/docs/introduction` official Neon docs.
2. `https://www.postgresql.org/docs/` official Postgres manual.
3. `https://swagger.io/docs/` OpenAPI / Swagger official site.
4. `https://en.wikipedia.org/wiki/Express.js` details on Express framework.
5. `https://www.npmjs.com/package/nodemon` nodemon npm page.
6. `https://www.youtube.com/watch?v=IlPPDIRjBKs` YouTube video tutorial.
7. `https://www.youtube.com/watch?v=SccSCuHhOw0` quick Express.js tutorial.

8. https://www.youtube.com/watch?v=dhMlXoTD3mQ Swagger video.
9. https://www.youtube.com/watch?v=f2EqECiTBL8 comprehensive Node.js + Express course.
10. https://youtu.be/9ZixI8Xy0T0?si=8YIfUflRKnW7CmtN - postgres quick Overview

3.3 Testing Approach

[Testing philosophy and coverage goals]

We aim to best the APIs using vitest because it fast and simple. while for frontend e2e testing os more reliable. I covers most cases and most integration.

3.3.1 Unit Testing

[Testing tools and methodology] Backend - vitest

3.3.2 End-to-End Testing

[UI automation strategy] Cypress - frontend testing Replaced by Playwright due to the multi window feature

3.3.3 Industry Resources

[Testing ROI studies or standards] https://testgrid.io/blog/cypress-testing/ https://www.testim.io/blog/cypress-vs-selenium/ https://dev.to/walmyrlimaesilv/10-reasons-why-you-should-use-cypress-for-web-testing-automation-jlb https://www.lambdatest.com/blog/cypress-vs-playwright/ https://www.browserstack.com/guide/playwright-vs-cypress https://www.checklyhq.com/learn/playwright/playwright-vs-cypress/ https://testgrid.io/blog/cypress-vs-playwright/

3.4 Development Tools

[Team development workflow]

3.4.1 Version Control

[Git workflow and branching model]

3.4.2 CI/CD Pipeline

[Automation tools and stages]

3.4.3 Hosting Platform

Backend hosted on Render

3.4.4 Industry Resources

[Reliability engineering standards] https://www.conventionalcommits.org/
en/v1.0.0/ https://blog.devgenius.io/make-a-meaningful-git-commit-message-with-semantic-commit-me

Chapter 4

Git Methodology

4.1 Overview

Git is a distributed version control system designed to handle everything from small to very large projects with speed and efficiency. It allows multiple developers to work on the same codebase simultaneously without interfering with each other's work. This chapter outlines the Git strategies, branching models, and versioning conventions recommended for your project. By adopting a consistent methodology, your team can improve collaboration, track changes effectively, and maintain a stable codebase throughout the development lifecycle.

4.2 Methodology

A well-defined Git workflow is essential for team coordination and code management. The following practices are recommended:

Branching Strategy

- **Main Branch**: The `main` branch contains production-ready code. It should always be stable and deployable.
- **Development Branch**: The `develop` branch serves as an integration branch for features. It is used to merge completed features before they are moved to `main`.
- **Feature Branches**: For each new feature, create a branch from `develop` named `feature/<feature-name>`. This keeps work isolated until the feature is complete and tested.
- **Release Branches**: When preparing a release, create a `release/<version>` branch from `develop`. This allows for final testing and bug fixes without disrupting ongoing development.

- **Hotfix Branches**: For urgent fixes in production, create a `hotfix/<issue>` branch from `main`. Once fixed, merge it into both `main` and `develop`.

Versioning Convention

For this project, a Calendar Versioning (CalVer) approach is recommended due to its simplicity and relevance for tools with frequent iterations:

- **Format**: `YYYY.MM.DD_MODIFIER`
- **Example**: `2025.08.12_sprint1`
- This format provides clarity on the timeline and context of the release (e.g., which sprint it corresponds to).

Commit Practices

- Write clear, descriptive commit messages.
- Use the imperative mood (e.g., "Add login feature" instead of "Added login feature").
- Commit often to ensure changes are incremental and traceable.

4.3 Why This Git Approach is Recommended

This Git methodology is ideal for your team and project for the following reasons:

- **Simplicity**: The branching strategy is easy to understand and implement, even for teams new to Git.
- **Flexibility**: Feature branches allow developers to work independently without affecting the main codebase.
- **Traceability**: CalVer provides clear, meaningful version numbers that reflect development cycles.
- **Scalability**: The workflow supports both small and large teams, making it suitable for your current and future projects.
- **Industry Alignment**: This approach is widely used in open-source and commercial projects, ensuring best practices are followed.

4.4 Industry Resources & Justification

The proposed Git methodology and versioning convention are supported by industry standards and authoritative resources:

Git and Version Control

- Official Git Documentation: The definitive guide to Git commands and concepts.
- A Successful Git Branching Model: The classic article introducing Git-Flow, which inspired the branching strategy recommended here.

Versioning Schemes

- Semantic Versioning (SemVer): A widely adopted scheme for versioning software based on meaningful version numbers.
- Calendar Versioning (CalVer): A versioning scheme based on project release dates, ideal for projects with time-based releases.
- Designing a Version: A practical guide on choosing and designing a versioning scheme, supporting the use of modifiers like `_sprint1`.

Chapter 5

Frontend Chapter

Your chapter content goes here.

5.1 First Section

5.1.1 A Subsection

Team Members: Dimpho Matea, Lebo K, Sinethemba K

Dimpho Mathea

Description: I implemented the API endpoints for CRUD regarding resources,my task was to ensure that when resources are posted the file url as well as the picture url and some other meta data is stored on the Neon DB ,these could not be possible if cloudinary could not be setup as a result of Neon db not being able to store Filesand images directly,I had faced multiple challenges were I had to familiarize myself with how Swagger API documentation works as well as the framework the team chose to adopt with regards to the backend ,the resource API was not working and is currently being monitored so as to provide efficient results when needed by the Frontend team.

Lebo

Description: I was writing the functions that act as middle ware for the Frontend and backend. However I was also mostly Responsible for the testing of the different parts of the Frontend. I setup and wrote all the test e2e style and also had to choose a framework that suits our application.

Sinethemba K

Description: Led the design of the web application, defining the visual style and user interface, and implemented layouts and styling using CSS to ensure a responsive and polished frontend. Responsible for wireframes and prototype using Lovable AI. Main challenges included defining the vision for the app and the look and feel for the best visual user experience, which entailed choosing

colors that complement each other without being overly stimulating. Translated images and wireframes into code, making elements responsive, testing responsiveness across multiple viewports, and writing code to accommodate each case.

Chapter 6

Backend Chapter

6.1 First Section

6.1.1 A Subsection

Team Members: Favour Mtshali, TrustGod M, Hulisani Innocent Netshaulu

Favour Mtshali

Description: I implemented the API endpoints for CRUD regarding resources,my task was to ensure that when resources are posted the file url as well as the picture url and some other meta data is stored on the Neon DB ,these could not be possible if cloudinary could not be setup as a result of Neon db not being able to store Filesand images directly,I had faced multiple challenges were I had to familiarize myself with how Swagger API documentation works as well as the framework the team chose to adopt with regards to the backend ,the resource API was not working and is currently being monitored so as to provide efficient results when needed by the Frontend team.

TrustGod

Description: I designed the database schema and handled authentication logic.

Huli

Description: I implemented the api endpoints for manual authentication, social media groups functionality and as well as the followers functionality(friends). My task was to allow users to authenticate using their emails and passwords as an alternative to using their google accounts, When i was implementing the authentication i had to resort to using the FireBase 3rd party authenticater because i had difficulty in using google for manual authentication, thus altering the original tech stack we planned on using initially.

Chapter 7

Testing Methodology

7.1 Testing Setup Process

This section details the step-by-step process for the testing Methodology for Study Buddy Enhanced.

7.2 Testing Strategy

To test the app we have choosen to do End-to-End testing because it covers more Integration that unit testing. Each unit can work alone does not ensure that it works well in context of the app.

7.3 Test Cases

We will be testing the API as they are and also seeing how they hold up against spam calls/requests. We will be doing End-to-End for all the part of the app a real user might use and how they will actually use it.

7.4 Continuous Integration

We have setup a auto test system on every pull request and every merge with the main branch on both backend and frontend. This allows the barnch to be protected at all costs.

7.5 Industry Resources

[Testing standards] `https://medium.com/@louisyoong/cypress-with-react-e2e-vs-component-testing-e20c2`
`https://bugbug.io/blog/software-testing/unit-testing-vs-end-to-end-testing/`

https://momentic.ai/resources/cypress-component-testing-vs-e2e-testing-a-deep-dive-for
https://www.globalapptesting.com/blog/unit-testing-vs-end-to-end-testing

Chapter 8

Meetings Chapter

8.1 Sprint One

8.1.1 Meeting One - Planning

Date: 7 August 2025
Attendees: Backend Team
Discussion: Roles on the backend and effective communication of the structure.
Minutes:

- **Roles Discussion**
  - Discussed the roles that need to be shared among members.
  - Highlighted that one member had too many responsibilities, which hindered effective work distribution and collaboration.
  - **Favour:** RESOURCE tables and file sharing system.
  - **Huli:** Caching of API and group-centric tables.
  - **Trust:** Picture-based files and backend setup.
- **Structure**
  - Emphasized the importance of core files: `models`, `index`, and `server`.
  - Used a platform to visualize the database schema.
- **Tasks and Objectives**
  - Members need to research and become comfortable with the provided tech stack.
  - Proper documentation required for reasoning and comparisons to alternative tools.

8.1.2 Meeting Two - Collaboration

Date: 13 August 2025

Attendees: Backend Team

Discussion: Cooperation with the independent developer team and integration of their API service.

Minutes:

- **Database Updates & Brainstorming Session**
  - Discussed how to integrate a third-party application that manages events to support the organization of study sessions.
  - Added new database tables to track information relevant to study sessions, specifically for use by group members.

8.1.3 Meeting Three - Rubric Review and Coordination

Date: 16 August 2025

Attendees: Backend & Frontend Teams

Discussion: Review of rubric checklist and acknowledgement of successfully completed items.

Minutes:

- **Task Review**
  - Agreed on which tasks are considered complete and which remain incomplete.
  - Reviewed the rubric and identified tasks to be implemented before the Sprint 1 deadline.
  - Discussed app components that are not functioning properly.
  - Identified actions to address the issues.
- **Authentication Fault**
  - Backend and frontend authentication failures were raised.
  - Planned further debugging of server-side authentication.
- **API Development Strategy**
  - Agreed that all API routes will be built based on client needs, not developer assumptions.
- **Branch Creation Methodology**
  - Allow individual merges to the version release branch.
  - Only approved merges to the main branch are permitted.
- **LaTeX Documentation**
  - All members are expected to update the shared LaTeX document on Overleaf for each feature they create.

8.1.4 Meeting Four - Rubric and LaTeX Documentation

Date: 18 August 2025
Attendees: Backend & Frontend Teams
Discussion: Review of rubric checklist, completed items, and LaTeX documentation.
Minutes:

- **LaTeX Platform**
  - Each member logged in and updated the shared LaTeX file.
- **Sprint 1 Rubric Review**
  - Reviewed the rubric and identified required implementations that were missed for Sprint 1.
- **API Requirements from Event Management Group**
  - Groups table will only share `venueID`, `Name`, and `Location`.
  - Created a separate table for the date and time of events.
- **GitHub Committing**
  - Implement semantic committing from now on.

8.2 Sprint Two

8.2.1 Meeting One - Backend Problem Resolution

Date: 31 August 2025
Attendees: Backend Team
Discussion: Discussion on how the APIS work as there were problems
Minutes:

- **API Dissection**
  - Discussed the roles that certain APIs had within building the application.
  - Highlighted the fact that Using swagger to document the API was a challenge ,as a team member that had the responsibilities of a certain API, the API was not working when it was called.
  - **Favour:** The team member with the struggle of making the API work.
  - **Trust:** The Backend facilitator,who help with understanding the pitfalls on the API not working.

8.2.2 Meeting Two - Retrospective

Date: 1 September 2025

Attendees: The whole Team

Discussion: Discussion how the sprint went and some resolutions for the following sprints

Minutes:

- **API Dissection**
  - Discussed the problematic integration of the API with the frontend as some were not working for days and it hindered progress.
  - Discussed the impact that us not having meetings was a problem as some team members were working blindly and there was no communication that was solid except on whatsApp when a problem arouse we solved it through text and this limited productivity.
  - **Resolutions:** The team members all agreed to try and improve the amount of times we meet and have a fixed schedule for meetings that will not change unless there is an emergency.Smaller meetings to be had with backend team if there is an issue with the backend and not involving the frontend team as this will waste time.Team Discussed the importance of reading messages that are communicated on whatsApp thoroughly as they may contain intel and minimise confusion.
  - **Reasons of Problems:**The team didnt meet as a result of this week being hectic with regards to submissions and there being a test written that needed undivided attention.This truly messed up the teams productivity in balancing and finishing the work in a timely manner.Some days were reserved as some team members have religious practices like the saddath day as well as bible study to uphold,these could have been used to push work thoroughly.

8.3 Sprint Three

8.3.1 Meeting One - Planning

8.3.2 Meeting Two - Retrospective

8.4 Sprint Four

8.4.1 Meeting One - Planning

8.4.2 Meeting Two - Final Retrospective

Chapter 9

Pitfall Chapter

Your chapter content goes here.

9.1 First Section

9.1.1 A Subsection

Pitfall: Documentation

Description: During development, we documented things but not in order that makes sense and we left out a lot of things that were truly important.

Resolution: Each member was instructed that when documenting on Papeeria to ensure that errors are mitigated immediately and not left for other people to solve.

Pitfall: API connection to Frontend

Description: There were multiple APIs that were built and not working so when the Frontend needed them they found out that they were not working,hence curbing productivity this is also as a result of the Backend having multiple issues.

Resolution: Had small meeting with the Backend to ensure smooth running APIs as well as agreed that they should be tested before the Frontend team uses them.

Pitfall: External API not ready

Description: Initially we had a team building the Event management application that agreed to integrate their API with our team after some days they reached out and informed us that they cannot provide the API we need so that we can integrate our application.

Resolution: The team will continue to look for alternative teams that can integrate our API with our application or change the way in which we need the API incase we do not in a timely manner.

Chapter 10

User Feedback

10.1 Overview

This chapter presents feedback from users who interacted with Study Buddy. It highlights the users, experiences, perceptions, and suggestions. The goal is to understand how the application performs in real scenarios, what users like, and areas for improvement.

10.2 Purpose of the Chapter

The purpose of this chapter is to:

- Summarize user experiences.
- Identify strengths and weaknesses of Study Buddy.
- Provide actionable insights for future improvements.

10.3 User Feedback Responses

Name: Devine Mtshali
Description: Devine found the application intuitive and easy to navigate. He appreciated the clean interface and the responsiveness of the features.They highlighted how the landing page as the app is opened was fascinating to them.

Name: Daniel
Description: Daniel mentioned that the application was fast and efficient. He suggested adding more tutorial guidance for first-time users,for example a clean seamless walkthrough of what the buttons do as well as what to expect from the pages and how to use them instead of using the intuition.

Name: Khanyisile Ndlovu
Description: Khanyisile mentioned the notifications feature and thought the

linked course recommendations would be very helpful. She requested more customization options.

Name: Thato Shandu
Description: Thato found the application visually appealing but suggested improving the feedback forms to be more user-friendly.They would like to see the fully made application as well as the holistic usage it provides.

Name: Simphiwe Mathe
Description: Simphiwe praised the stability of the application but recommended adding an FAQ or help section to assist new users.

10.4 Summary of Suggested Improvements

Based on the user feedback collected, several areas for improvement and additional features have been identified. Users appreciated the intuitive interface, responsiveness, and the stability of the application, but they highlighted the need for more guidance and enhanced usability. Suggested improvements include adding a clean and seamless walkthrough for first-time users, providing detailed explanations for the buttons and pages, enhancing the feedback forms to be more user-friendly, offering more customization options for notifications and course recommendations, and including an FAQ or help section to assist new users.

This feedback was collected by sharing the deployed application link with friends and colleagues, allowing them to interact with the live system. Users were asked to explore the application and provide honest opinions on their experience, which enabled the team to gather actionable insights for further development and refinement of the application.